

Deep Learning (IST, 2023-24)

Homework 2

André Martins, Fábio Vital, João Santinha

Deadline: Saturday, January 6, 2024.

Please turn in the answers to the questions below in a PDF file, together with the code you implemented to solve them (when applicable).

IMPORTANT: Please write 1 paragraph indicating clearly what was the contribution of each member of the group in this project. A penalization of 10 points will be applied if this information is missing.

Please submit a **single zip file** in Fenix under your group's name.

Question 1 (30 points)

1. (5 points) Consider the self-attention layer of a transformer with a single attention head, which performs the computation $\mathbf{Z} = \text{Softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$, where $\mathbf{Q} \in \mathbb{R}^{L \times D}$, $\mathbf{K} \in \mathbb{R}^{L \times D}$, and $\mathbf{V} \in \mathbb{R}^{L \times D}$, for a sequence length L and hidden size D . What is the computational complexity, in terms of L , of computing $\mathbf{Z}$? Briefly explain why this could be problematic for long sequences.

Solution: The complexity is $O(L^2)$ due to the matrix multiplication $\mathbf{Q}\mathbf{K}^\top$ inside the softmax. This can be problematic if L is very large.

2. (10 points) We will show that by suitably approximating the softmax transformation the computational complexity above can be reduced. Consider the McLaurin series expansion for the exponential function,

$$\exp(t) = \sum_{n=0}^{\infty} \frac{t^n}{n!} = 1 + t + \frac{t^2}{2} + \frac{t^3}{6} + \dots$$

Using only the first three terms in this series, we obtain $\exp(t) \approx 1 + t + \frac{t^2}{2}$. Use this approximation to obtain a feature map $\phi : \mathbb{R}^D \rightarrow \mathbb{R}^M$ such that, for arbitrary $\mathbf{q} \in \mathbb{R}^D$ and $\mathbf{k} \in \mathbb{R}^D$ we have $\exp(\mathbf{q}^\top \mathbf{k}) \approx \phi(\mathbf{q})^\top \phi(\mathbf{k})$. Express the dimensionality of the feature space M as a function of D . What would be the dimensionality if you used $K \geq 3$ terms in the McLaurin series expansion (as a function of D and K)?

Solution: Let $\tilde{\exp}(t) = 1 + t + \frac{t^2}{2}$. We have $\tilde{\exp}(\mathbf{q}^\top \mathbf{k}) = 1 + \mathbf{q}^\top \mathbf{k} + \frac{(\mathbf{q}^\top \mathbf{k})^2}{2} = 1 + \mathbf{q}^\top \mathbf{k} + \frac{\text{Tr}(\mathbf{q}\mathbf{q}^\top \mathbf{k}\mathbf{k}^\top)}{2} = 1 + \mathbf{q}^\top \mathbf{k} + \frac{\text{vec}(\mathbf{q}\mathbf{q}^\top)^\top \text{vec}(\mathbf{k}\mathbf{k}^\top)}{2} = \phi(\mathbf{q})^\top \phi(\mathbf{k})$, with $\phi(\mathbf{x}) = [1, \mathbf{x}, \text{vec}(\mathbf{x}\mathbf{x}^\top)/\sqrt{2}]$. The dimensionality of the feature space is $1 + D + D^2$. With $K \geq 3$, we would need $(K-1)$ -ary feature interactions, so we would have $M = O(D^{K-1})$.

3. (10 points) Using the approximation $\exp(\mathbf{q}^\top \mathbf{k}) \approx \phi(\mathbf{q})^\top \phi(\mathbf{k})$, and denoting by $\Phi(\mathbf{Q}) \in \mathbb{R}^{L \times M}$ and $\Phi(\mathbf{K}) \in \mathbb{R}^{L \times M}$ the matrices whose rows are (respectively) $\phi(\mathbf{q}_i)$ and $\phi(\mathbf{k}_i)$, where $\mathbf{q}_i$ and $\mathbf{k}_i$ denote (also respectively) the i^{th} rows of the original matrices $\mathbf{Q}$ and $\mathbf{K}$, show that the self-attention operation can be approximated as $\mathbf{Z} \approx \mathbf{D}^{-1} \Phi(\mathbf{Q}) \Phi(\mathbf{K})^\top \mathbf{V}$, where $\mathbf{D} = \text{Diag}(\Phi(\mathbf{Q}) \Phi(\mathbf{K})^\top \mathbf{1}_L)$ (here, $\text{Diag}(\mathbf{v})$ denotes a matrix with the entries of vector $\mathbf{v}$ in the diagonal, and $\mathbf{1}_L$ denotes a vectors of ones with size L).

Solution: Since $\text{softmax}(\mathbf{z}) = \exp(\mathbf{z}) / \mathbf{1}^\top \exp(\mathbf{z})$, using the approximation $\exp(\mathbf{q}^\top \mathbf{k}) \approx \phi(\mathbf{q})^\top \phi(\mathbf{k})$ we obtain $\mathbf{Z} \approx \mathbf{D}^{-1} \Phi(\mathbf{Q}) \Phi(\mathbf{K})^\top \mathbf{V}$.

4. (5 points) Show how we can exploit the above approximation to obtain a computational complexity which is linear in L . How does the computational complexity depend on M and D ?

Solution: To compute $\mathbf{D}$, we can use the associative property of the matrix multiplication and compute $\Phi(\mathbf{K})^\top \mathbf{1}_L$ first, which has cost $O(ML)$ and leads to a vector of size M . Left multiplying by $\Phi(\mathbf{Q})$ has cost $O(LM)$ and inverting the diagonal matrix $\mathbf{D}$ costs $O(L)$, so the total cost to obtain $\mathbf{D}^{-1}$ is $O(LM)$. Using the associative property of the matrix multiplication, we can perform the matrix multiplication $\Phi(\mathbf{K})^\top \mathbf{V}$ first, which has cost $O(MLD)$ and leads to a matrix $M \times D$, then left multiply by $\Phi(\mathbf{Q})$, which costs $O(LMD)$ and leads to a matrix $L \times D$, then left multiply by $\mathbf{D}^{-1}$, which costs $O(LD)$. Therefore, the total cost is $O(LMD)$, which is linear in L , M , and D .

Question 2 (35 points)

Image classification with CNNs. In this exercise, you will implement a convolutional neural network to perform classification using the OCTMNIST dataset.

As previously done in Homework 1, you will need to download the OCTMNIST dataset. You can do this by running the following command in the homework directory:

```
python download_octmnist.py
```

Python skeleton code is provided (hw2-q2.py). You will now try out convolutional networks. For this exercise, we recommend you use a deep learning framework with automatic differentiation (suggested: Pytorch).

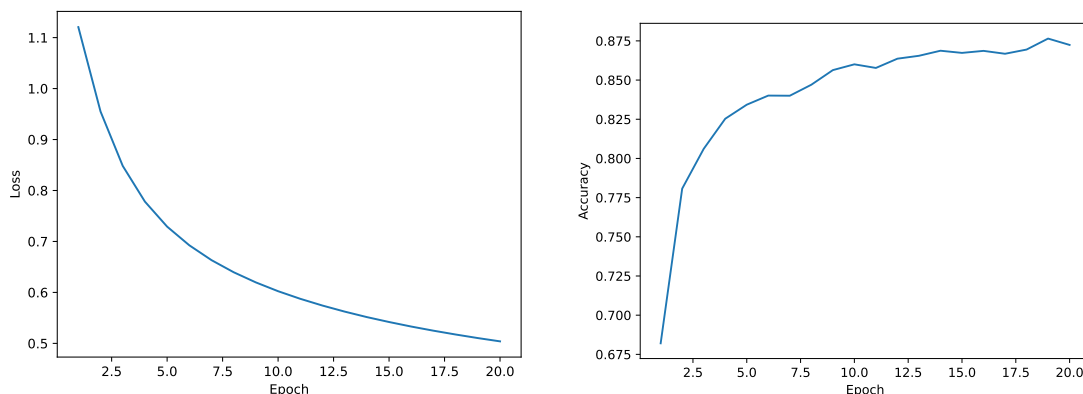
- (20 points) Implement a simple convolutional network with the following structure:
 - A convolution layer with 8 output channels, a kernel of size 3x3, stride of 1, and padding chosen to preserve the original image size. Note: Assign convolution layer initialization to a variable called `self.conv1`.
 - A rectified linear unit activation function.
 - A max pooling with kernel size 2x2 and stride of 2.
 - A convolution layer with 16 output channels, a kernel of size 3x3, stride of 1, and padding of zero.
 - A rectified linear unit activation function.
 - A max pooling with kernel size 2x2 and stride of 2.

- An affine transformation with 320 output features (to determine the number of input features use the number of channels, width and height of the output of the second block. Hint: The number of input features = *number of output channels* $\times$ *output width* $\times$ *output height*).
- A rectified linear unit activation function.
- A dropout layer with a dropout probability of 0.7.
- An affine transformation with 120 output features.
- A rectified linear unit activation function.
- An affine transformation with the number of classes followed by an output LogSoftmax layer.

Hint: use the functions `nn.Conv2d` and `nn.MaxPool2d`.

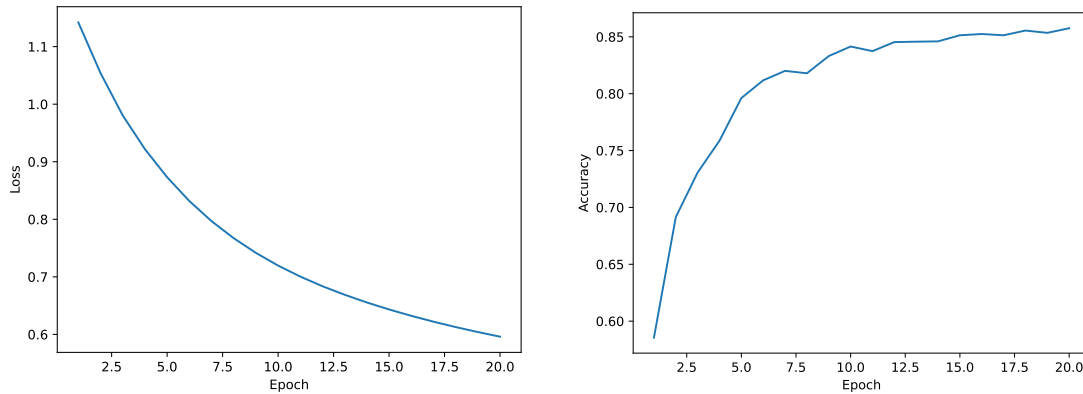
Train your model for 15 epochs using SGD tuning only the learning rate on your validation data, using the following values: 0.1, 0.01, 0.001. Report the learning rate of best configuration and plot two things: the training loss and the validation accuracy, both as a function of the epoch number.

Solution: Training 20 epochs with a learning rate of 0.01, a l2 weight decay of 0, a dropout probability of 0.7 and using the SGD optimizer we get a validation accuracy of 0.8724 and a test accuracy of 0.8355.



- (10 points) Implement and asses a similar network but where the max-pooling layers were removed and `self.conv1` and `self.conv2` are different. `self.conv1` is a convolution layer with 8 output channels, a kernel of size 3x3, padding of 1 and stride of 2, and `self.conv2` is a convolution layer with 16 output channels, a kernel of size 3x3, padding of 0, and stride of 2. Modify the `__init__` and the `forward` functions to use the `no_maxpool` variable and ensure the ability to switch between current and previous definitions of the `self.conv1` and `self.conv2` layers and application or not of the max-pooling layer. Report the performance of this network using the optimal hyper-parameters defined in the previous question.

Solution: Training 20 epochs with a learning rate of 0.01, a l2 weight decay of 0, a dropout probability of 0.7, no max-pooling layers, and using the SGD optimizer we get a validation accuracy of 0.8576 and a test accuracy of 0.8166.



3. (5 points) Implement the function `get_number_trainable_params` to determine the number of trainable parameters of CNNs from the two previous questions. What justifies the difference in terms of performance between the networks?

Solution: The number of trainable parameters is the same for the two CNNs (224892). With the same capacity, the justification for the difference found in terms of performance for the two networks is the use of the max-pooling layers that improve invariance ensuring that if there is a small translation of the input, the values of the pooled outputs remain unchanged.

Question 3 (35 points)

Automatic Speech Recognition Automatic speech recognition (ASR) is a deep-learning application that enables the automatic transcription and translation of spoken language into text. The task you will address is the transcription of the LJ Speech dataset (v 1.1). This small dataset has around 13000 short audio clips and the corresponding transcription of reading passages from several books. You will use and compare two different encoder-decoder architectures to solve this problem. Figure 1 depicts a general overview of an encoder-decoder architecture for the ASR application.

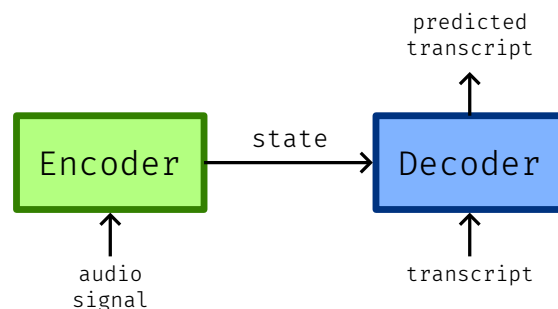


Figure 1: Encoder-decoder architecture to solve ASR task. The encoder receives a processed version of an audio signal and outputs a hidden representation. The decoder receives and merges the received transcript (sequence of tokens) and the encoder's output, predicting the next token of each input token.

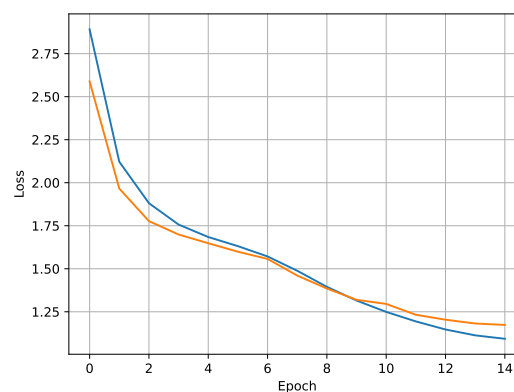
The challenge proposed in questions 1 and 2 aims to implement the forward pass of different decoder implementations and compare their performance. We will skip the definition of the

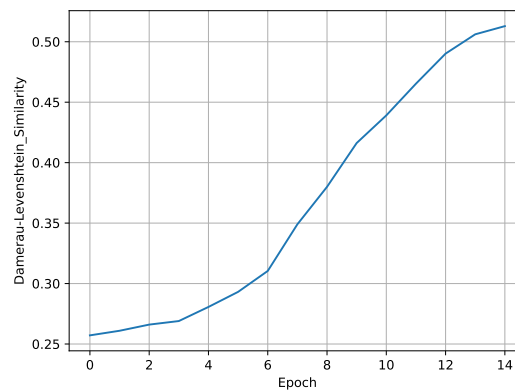
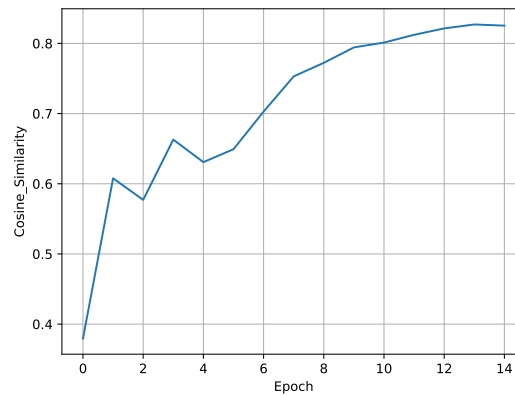
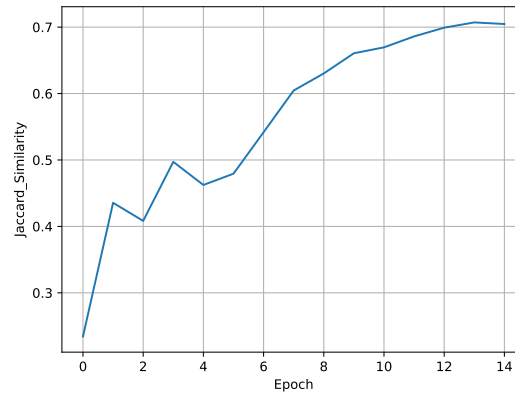
encoder's architecture since the entire implementation is already present in the code provided (hw2-q3.ipynb). For completeness, the encoder's architecture appears as supplementary material in Appendix B. In question 1, the decoder employs a recurrent-based approach to process the input, whereas an attention-based mechanism is used in question 2. Both decoders receive as input the text to decode as a sequence of tokens and the encoder's output. For each input token, the decoder's output will be a the logits corresponding to the probability of choosing each token in the vocabulary as the next token in the sequence.

1. (10 points) The decoder structure to employ is the following (see also Figure 2a):
 1. Embedding layer to convert the input text tokens into dense vectors, also called token embeddings.
 2. The token embeddings pass through a normalization layer followed by an LSTM.
 3. A residual cross-attention block will merge and attend the LSTM's output with the encoder's output. The query input will receive the LSTM's output, and the key and value inputs receive the encoder's output. For more details, refer to Appendix C.
 4. Normalize the residual block output using a normalization layer.
 5. The output of the attention mechanism passes through a single linear layer (classifier) to scale the hidden representation to the desired output dimension, in this case, the vocabulary size $|V| = 35$.

Go to the notebook and implement the `_forward` method of the `TextDecoderRecurrent` class. Note that every layer is already initialized in the `TextDecoderRecurrent`'s `__init__` method. Next, run the cell under the markdown cell (a). Plot the comparison of the training and validation loss over epochs and the string similarity scores obtained in the validation set. Also, report the final test loss and the string similarity scores obtained with the test set.

Solution:





Final test loss: 1.183

Final test jaccard similarity: 0.715

Final test cosine similarity: 0.832

Final test Damerau-Levenshtein similarity: 0.509

2. (10 points) The decoder structure to employ is the following (see also Figure 2b):

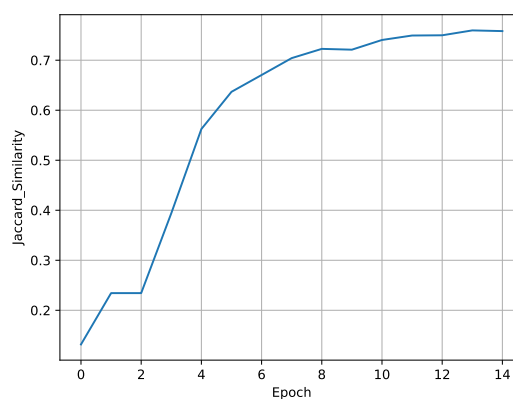
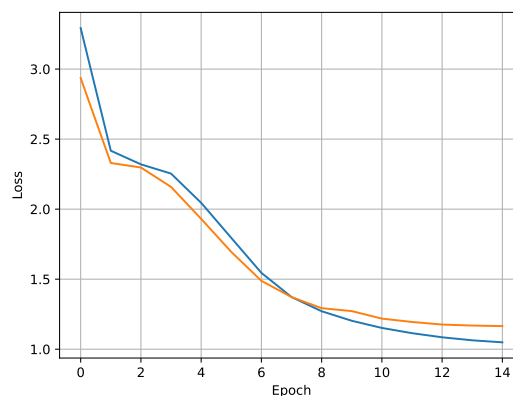
1. Embedding layer to convert the input text tokens into dense vectors.
2. Add a position embeddings matrix to the token embeddings.
3. The token embeddings pass through a residual attention layer as input for the queries, keys, and values. Additionally, this layer receives a masking matrix to ensure that the

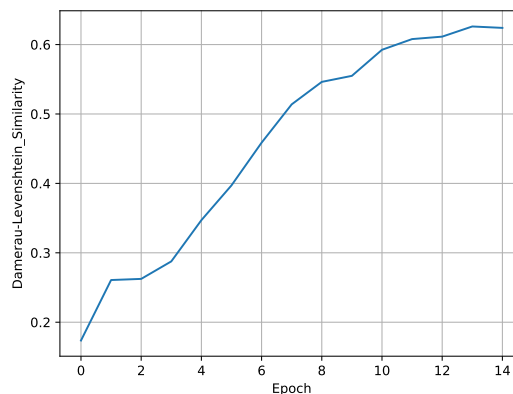
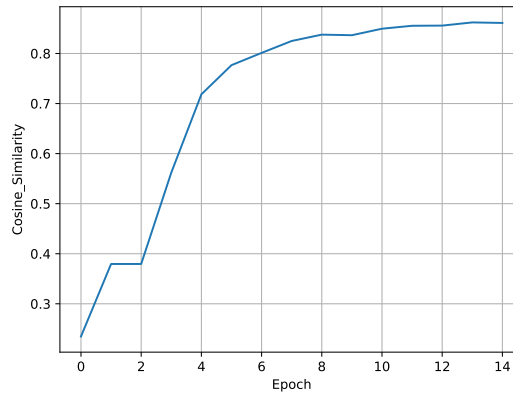
prediction of the current token does not attend to future tokens in the input. For more details, refer to Appendix A.

4. A residual cross-attention block will merge and attend the previous attention's output with the encoder's output. The query input will receive the attention layer output, and the key and value inputs receive the encoder's output. For more details, refer to Appendix C.
5. Normalize the residual block output using a normalization layer.
6. The output of the normalization layer passes through a single linear layer (classifier) to scale the hidden representation to the desired output dimension, in this case, the vocabulary size $|V| = 35$.

Go to the notebook and implement the `forward` method of the `TextDecoderTransformer` class. Note that every layer is already initialized in the `TextDecoderTransformer`'s `__init__` method. Next, run the cell under the markdown cell (a). Plot the comparison of the training and validation loss over epochs and the string similarity scores obtained in the validation set. Also, report the final test loss and the string similarity scores obtained with the test set.

Solution:





Final test loss: 1.160

Final test jaccard similarity: 0.765

Final test cosine similarity: 0.866

Final test Damerau-Levenshtein similarity: 0.634

- (10 points) Comment on the differences in the test results obtained using the decoder architectures in questions 1 and 2. *Note: start by illustrating how the LSTM (in question 1) and the attention mechanism (in question 2) process the text input.*

Solution: The final test loss is smaller when using the second decoder architecture, a fully attention-based architecture. In the first architecture, the decoder uses an LSTM to compute a hidden representation of the text input. In RNNs, the tokens are processed sequentially, meaning the hidden state is also updated sequentially. As such, the encoding of a specific token strongly affects the representation of the next few tokens, but its influence quickly weakens as the sequence progresses. Consequently, it is hard to have long dependencies in tokens using RNNs. On the other hand, attention-based mechanisms do not suffer from this problem since each token can be accessed in constant time. Therefore, the sequences are processed as a whole (rather than token by token), which avoids long-term dependency issues.

- (5 points) Comment on the score values obtained by each string similarity score used. Why each score has different values based on what each similarity tries to evaluate?

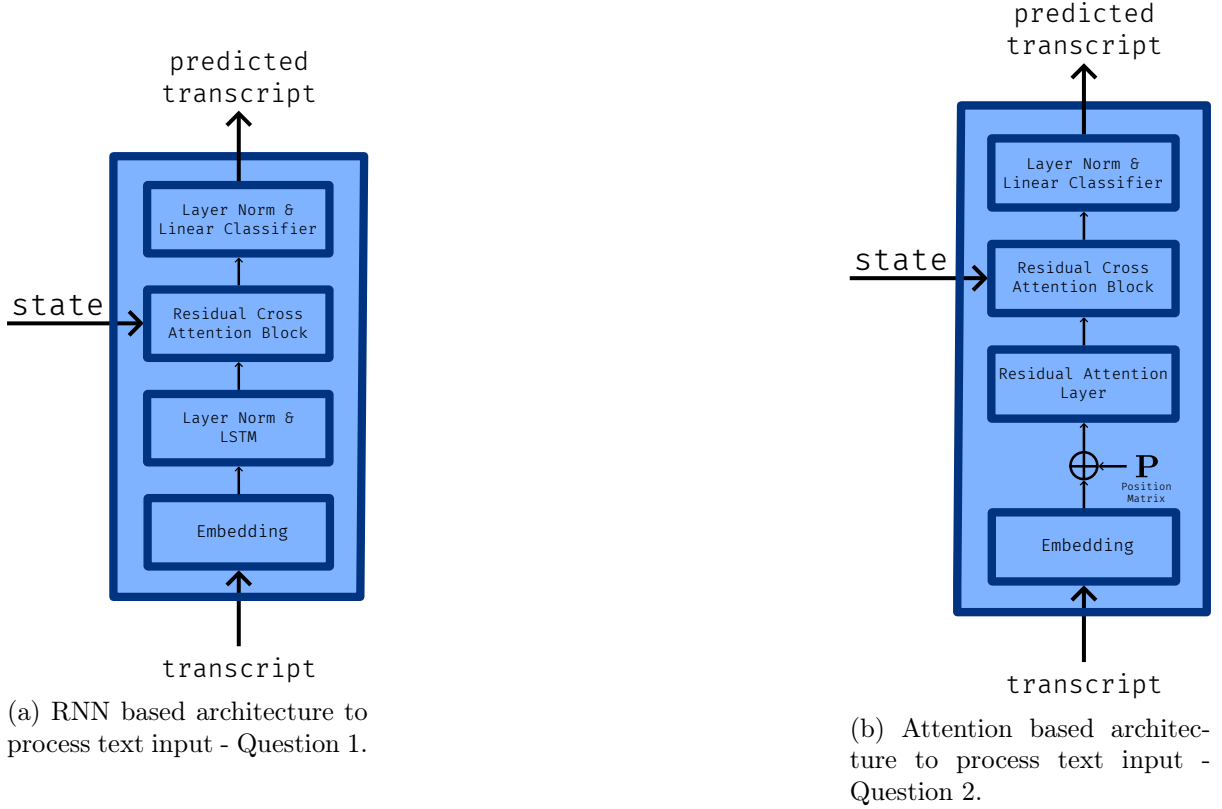


Figure 2: Decoder architectures to implement.

Solution: Jaccard and cosine similarities are token-based algorithms, which compare strings based on their units (in our case, characters). The jaccard score considers strings as sets, comparing their similarity by taking the ratio between shared and combined tokens. The cosine similarity transforms strings into numerical vectors to output the normalized angle between the strings. Token-based algorithms emphasize presence rather than order, which can be misleading in a speech recognition task since the objective is to get the exact transcription of the speech signal.

The Damerau-Levenshtein similarity is an edit-based algorithm which tries to compute the number of operations to transform one string into another. The operations are insertion, deletion, substitution, and transposition. This similarity has the lowest score, around 73% for the second architecture. With this score, we observe that the generated transcript still has some single-character differences when compared to the original text, which can be sufficient to decrease the quality of the transcript from a spell-checking perspective.

A Multi-Head Attention Mechanism & Residual Attention Layer

The attention mechanism used in both encoder and decoder architectures is the *scaled dot-product* attention, defined as $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) := \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}})\mathbf{V}$. Normally, the queries, keys, and values are packed together into matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, respectively. Furthermore, instead of applying a single attention pass, this function is parallelized k times, where each attention head computes a different pass. To maintain the same output dimension, the output

of each head is concatenated and projected using a single linear layer into the desired dimension.

$$\text{MultiHeadAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V} := \text{Concat}(\text{head}_1, \dots, \text{head}_k) \mathbf{W}^O \\ \text{where head}_i = \text{Attention}(\mathbf{Q} \mathbf{W}_i^Q, \mathbf{K} \mathbf{W}_i^K, \mathbf{V} \mathbf{W}_i^V).$$

If necessary, the projections $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, and $\mathbf{W}_i^V$ reduce or increase the queries, keys, and values dimensions, respectively. $\mathbf{W}^O$ is the linear layer that reduces the dimensionality of the concatenated outputs from each head.

A residual attention layer is simply the combination of the following layers: $\text{LayerNorm}(\mathbf{x} + \text{MultiHeadAttention}(\mathbf{x}', \mathbf{x}', \mathbf{x}'))$, where $\mathbf{x}' = \text{LayerNorm}(\mathbf{x})$.

B Encoder's Architecture

The encoder's architecture is based on the Transformer, comprising an embedding mechanism followed by a stack of $N_{\text{enc}} = 4$ identical blocks.

Since the encoder's input is an audio signal in the form of a mel-scale conversion of a spectrogram (frequency of a signal), the embedding mechanism starts by applying a sequence of two 1D convolutions, each followed by the GELU activation function. Both convolutions have a kernel size of 3 and padding of 1. Additionally, the second convolution reduces the time dimension (last dimension) by half by using a stride of 2. Afterward, add absolute sinusoidal position embeddings to the hidden representation before giving it to the stack of encoder layers.

Regarding the stack of encoder blocks, each block comprises two consecutive sub-blocks. All sub-blocks have the same design: the input passes through a predefined module, followed by applying layer normalization to the resulting sum of the module's output with a residual connection of the input, $\text{SubBlock}(\mathbf{x}) = \text{LayerNorm}(\mathbf{x} + \text{Module}(\text{LayerNorm}(\mathbf{x})))$. The modules for the first and second sub-layers are:

1. Multi-Head Attention layer, $\text{Module}(\mathbf{x}) = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{x}, \mathbf{x})$, where $\mathbf{x}$ is given as the layer's query, key, and value inputs, respectively. The hidden dimension is $h_{\text{enc}} = 200$ and the mechanism has $k_{\text{enc}} = 2$ heads.
2. Feed-forward network, $\text{Module}(\mathbf{x}) = \text{MLP}(\mathbf{x})$, having a single hidden layer, with the GELU function as the activation function. The hidden layer dimension is $h_{\text{enc}} \times 2$, and the output dimension is h_{enc} . GELU is a smooth variation of ReLU, $\text{GELU}(x) = xP(X \leq x) \approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$, where $X \sim \mathcal{N}(0, 1)$.

C Residual Cross-Attention Block

The residual attention block is a refined version of the attention-based blocks introduced in Appendix B. In the cross-attention design, each block has two consecutive sub-blocks. The second sub-block has the same architecture introduced in Appendix B, $\text{SubBlock}(\mathbf{x}) = \text{LayerNorm}(\mathbf{x} + \text{Module}(\text{LayerNorm}(\mathbf{x})))$. For the first sub-block, we have $\text{SubBlock}(\mathbf{x}, \mathbf{z}) = \text{LayerNorm}(\mathbf{x} + \text{Module}(\text{LayerNorm}(\mathbf{x}), \mathbf{z}))$. The modules employed for the sequence of two sub-blocks are as follows:

1. Multi-Head Attention layer, $\text{Module}(\mathbf{x}, \mathbf{z}) = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{z}, \mathbf{z})$, where $\mathbf{x}$ is the output of previous sub-block and given as the query. Typically, since the residual block appears inside the decoder, $\mathbf{z}$ is often the encoder's output.
2. Feed-forward network, $\text{Module}(\mathbf{x}) = \text{MLP}(\mathbf{x})$, having a single hidden layer, with the GELU function as the activation function.